Exercise 5

Joshua John E. Santiago

2026-11-06

#R Lab: Data Structures - Matrix and Array

#Part A — Creating Matrices

```
m1 <- matrix(1:9, nrow = 3, ncol = 3)
print(m1)
```

```
##      [,1] [,2] [,3]
## [1,]    1    4    7
## [2,]    2    5    8
## [3,]    3    6    9
```

```
m2 <- matrix(1:9, nrow = 3, ncol = 3, byrow = TRUE)
print(m2)
```

```
##      [,1] [,2] [,3]
## [1,]    1    2    3
## [2,]    4    5    6
## [3,]    7    8    9
```

```
# m1 fills down the columns (1, 2, 3 go in column 1), while m2 fills across
# the rows (1, 2, 3 go in row 1).
```

```
scores <- matrix(c(88, 92, 79, 95, 84, 90), nrow = 2, ncol = 3, byrow = TRUE)
rownames(scores) <- c("Quiz1", "Quiz2")
colnames(scores) <- c("Ana", "Ben", "Cid")

print(scores)
```

```
##       Ana Ben Cid
## Quiz1  88  92  79
## Quiz2  95  84  90
```

```
rownames(scores)
```

```
## [1] "Quiz1" "Quiz2"
```

```
colnames(scores)
```

```
## [1] "Ana" "Ben" "Cid"
```

```
dimnames(scores)
```

```
## [[1]]
## [1] "Quiz1" "Quiz2"
##
## [[2]]
## [1] "Ana" "Ben" "Cid"
```

#Question #1

```
# Answer: byrow = TRUE changes the order R fills the matrix. Normally R fills
# column by column, so 1, 2, 3 go down column 1. With byrow = TRUE it fills
# row by row, so 1, 2, 3 go across row 1. Same numbers and same size, just
# arranged differently.
```

#Question #2

```
m3 <- matrix(1:8, nrow = 3, ncol = 3)
```

```
## Warning in matrix(1:8, nrow = 3, ncol = 3): data length [8] is not a
## sub-multiple or multiple of the number of rows [3]
```

```
print(m3)
```

```
##      [,1] [,2] [,3]
## [1,]    1    4    7
## [2,]    2    5    8
## [3,]    3    6    1
```

```
# Answer: R still makes the matrix but runs out of numbers, so it goes back to
# the start and reuses 1 for the last slot. It also gives a warning that the
# data length does not match the matrix size. Since it is only a warning and
# not an error, it is easy to miss.
```

#Part B — Subsetting Matrices

```
# Single value in row 1, column 2
scores[1, 2]
```

```
## [1] 92
```

```
# Entire first row
scores[1, ]
```

```
## Ana Ben Cid
##  88  92  79
```

```
# Entire Ben column
scores[, "Ben"]
```

```
## Quiz1 Quiz2
##    92    84
```

```
# Values greater than or equal to 90
scores[scores >= 90]
```

```
## [1] 95 92 90
```

#Question #3

```
class(scores[1, ])
```

```
## [1] "numeric"
```

```
class(scores[, "Ben"])
```

```
## [1] "numeric"
```

```
# Answer: Both give numeric, so R turns a single row or column into a plain
# vector. It does this because one row or column only needs one dimension, and
# a vector is easier to use with things like mean() or sum().
```

#Question #4

```
scores[scores >= 90]
```

```
## [1] 95 92 90
```

```
# Answer: scores >= 90 just picks out the values that are TRUE, and those
# values come from different spots in the matrix. They do not form a full row
# or column anymore, so R returns a plain vector and the names are lost.
# scores[1, ] keeps the names because it takes a whole row.
```

#Part C — Matrix Properties and Operations

```
# Dimensions of scores
dim(scores)
```

```
## [1] 2 3
```

```
nrow(scores)
```

```
## [1] 2
```

```
ncol(scores)
```

```
## [1] 3
```

```
# Transpose
scores_t <- t(scores)
print(scores_t)
```

```
##      Quiz1 Quiz2
## Ana     88    95
## Ben     92    84
## Cid     79    90
```

```
dim(scores_t)
```

```
## [1] 3 2
```

```
a <- matrix(1:4, nrow = 2)
b <- matrix(5:8, nrow = 2)

# Element-wise addition
a + b
```

```
##      [,1] [,2]
## [1,]    6   10
## [2,]    8   12
```

```
# Element-wise multiplication
a * b
```

```
##      [,1] [,2]
## [1,]    5   21
## [2,]   12   32
```

```
# True matrix multiplication
a %*% b
```

```
##      [,1] [,2]
## [1,]   23   31
## [2,]   34   46
```

```
# Compare to a * b above. They are different.
a * b
```

```
##      [,1] [,2]
## [1,]    5   21
## [2,]   12   32
```

#Question #5

```
# Answer: a * b multiplies the numbers in the same position, so 1*5, 2*6, and
# so on. a %*% b is real matrix multiplication, where each result is a row of
# a times a column of b, added up. For example, the top left is 1*5 + 3*6 = 23.
# That is why the answers are different.
```

#Question #6

```
# Answer: When you transpose, rows become columns and columns become rows, so
# dim goes from 2 x 3 to 3 x 2. The names are labels for the rows and columns,
# so they have to move too. Otherwise Ana would be labeled with the wrong
# numbers.
```

#Part D — cbind() and rbind()

```
# Add Dex as a new column
scores_v2 <- cbind(scores, Dex = c(85, 91))
print(scores_v2)
```

```
##       Ana Ben Cid Dex
## Quiz1  88  92  79  85
## Quiz2  95  84  90  91
```

```
# Add Quiz3 as a new row (original 3 students only)
scores_v3 <- rbind(scores, Quiz3 = c(80, 88, 93))
print(scores_v3)
```

```
##       Ana Ben Cid
## Quiz1  88  92  79
## Quiz2  95  84  90
## Quiz3  80  88  93
```

#Question #7

```
cbind(scores, c(85, 91, 100))
```

```
## Warning in cbind(scores, c(85, 91, 100)): number of rows of result is not a
## multiple of vector length (arg 2)
```

```
##       Ana Ben Cid
## Quiz1  88  92  79 85
## Quiz2  95  84  90 91
```

```
# Answer: It gives a warning, not an error. The matrix has 2 rows but I gave 3
# values, so R only uses 85 and 91 and drops the 100. The code still runs, so
# you could miss it. cbind() and rbind() need matching sizes because a matrix
# is a rectangle, so every column needs the same number of rows.
```

#Question #8

```
# Answer: I would use rbind() when adding new records with the same columns.
# For example, in a table of daily sales where the columns are Coffee, Tea,
```

```
# and Juice, each new day is a new row. The columns stay the same and only the
# number of days grows.
```

#Part E — Arrays: Creating and Subsetting

```
arr <- array(1:12, dim = c(2, 3, 2))
print(arr)
```

```
## , , 1
##
##      [,1] [,2] [,3]
## [1,]    1    3    5
## [2,]    2    4    6
##
## , , 2
##
##      [,1] [,2] [,3]
## [1,]    7    9   11
## [2,]    8   10   12
```

```
# arr displays as two separate 2x3 slices, layer 1 has 1 to 6 and layer 2
# has 7 to 12.
```

```
# Row 1, column 2, second layer
arr[1, 2, 2]
```

```
## [1] 9
```

```
# Entire first layer
arr[ , , 1]
```

```
##      [,1] [,2] [,3]
## [1,]    1    3    5
## [2,]    2    4    6
```

```
dim(arr)
```

```
## [1] 2 3 2
```

#Question #9

```
# Answer: dim(arr) gives 2 3 2. The first number is the rows, the second is
# the columns, and the third is the number of layers (stacked matrices).
```

#Question #10

```
# Answer: Grades of students over different semesters. Rows are the students,
# columns are the subjects, and each layer is a semester. A single matrix has
# only rows and columns, so it could not keep each semester separate.
```

#Part F — Critical Thinking Challenge

```
class_a <- matrix(c(85, 90, 78, 92), nrow = 2, byrow = TRUE,
                  dimnames = list(c("S1", "S2"), c("Midterm", "Final")))
class_b <- matrix(c(88, 84, 95, 91), nrow = 2, byrow = TRUE,
                  dimnames = list(c("S3", "S4"), c("Midterm", "Final")))

# Combine the two classes (new students, so rbind)
all_students <- rbind(class_a, class_b)
print(all_students)
```

```
##    Midterm Final
## S1      85    90
## S2      78    92
## S3      88    84
## S4      95    91
```

```
# Average score of each student
rowMeans(all_students)
```

```
##   S1   S2   S3   S4
## 87.5 85.0 86.0 93.0
```

#Question #11

```
# Answer: I used rbind() because class_a and class_b have the same columns
# (Midterm and Final) but different students (S1, S2 and S3, S4). Different
# students means new rows. If the row names were the same, it would be cbind().
```

#Question #12

```
# Answer: I would stack all 5 matrices on top of each other using rbind(),
# since they all have the same 2 columns. Then I would check with dim(). The
# columns should still be 2, and the rows should equal the total number of
# students in all 5 classes. For example, 2 students per class gives 10 2.
```

#Conclusion

```
# Answer: A matrix has 2 dimensions (rows and columns), while an array can
# have 3 or more, like stacked matrices. Both can only hold one data type.
# cbind() and rbind() are useful for growing a dataset. For example, rbind()
# can add a new week of sales as a new row, and cbind() can add a new product
# as a new column.
```